

NEW ALGORITHMS FOR COMPARING SYMBOL SEQUENCES

Wen-Jing Hsu
Department of Computer Science
Michigan State University
East Lansing, Michigan

A framework is presented for computing substrings and subsequences shared between two symbol sequences. Efficient algorithms are then provided for determining the maximal common substrings.

I. INTRODUCTION

Genetic engineers must routinely perform a variety of nucleic acid and amino acid sequence analyses[Mode84]. Such analyses, in principle, can be carried out by hand. However, a symbol sequence (a *string*) in this context often consists of hundreds of symbols. For example, the amino acid sequence of an ARV-2 DNA [Sanc85], which is a human retrovirus usually associated with AIDS disease, consists of about 10 thousand symbols. Manual analysis on such long sequences may take an excessive amount of time. More seriously, humans may fail to recognize patterns over great lengths of sequences. To carry out the analyses on the computers, correct and efficient algorithms must be devised. Since efficient tools can be important in speeding up vital research, the objective of this study is to examine a string processing problem and provide provably efficient algorithms even when the input size (i.e. the length of the input strings) is large.

1.1. The String Matching Problems

A *common substring* (or, CS, for short) of two strings A and B is a string shared by both strings. A CS is *maximal* if it is not contained in any other common substring of A and B. For example, let $A = 'abcde'$ and $B = 'bcdeabf'$, then $'bcde'$, $'f'$, and $'ab'$ are the set of maximal common substrings (or, MCSs, for short) of A and B; while CS $'cd'$ and $'a'$ are not maximal.

The *Maximal Common Substring Problem* is to enumerate all of the maximal common substrings of two given strings. This problem falls in a more general category of problems described below [Aho74]. The following is only a partial list of them, which

may be broadly categorized into two subclasses.

In the first category, the location of a specified substructure in a string is of interest. For example,

1. The well-known *Substring Recognition Problem* [Knut74] is to find an instance or all instances of a known string (called Pattern) inside another (presumably longer) string.
2. *Matching Pattern with Don't Care Symbols* [Fisc74] is a generalized version of the above problem, where the pattern contains don't care (wild card) symbols.
3. *Multiple Pattern Matching Problem* [Aho74] is again similar to the previous problem. Here a set of patterns are given.

In the second category of problems, a string which is unknown beforehand is to be derived from the given strings. For example:

4. *Repeated Substrings Problem* : Given a string S , list all the repeated substrings of S [Karp72].
5. *Longest Prefixes Problem* : Given strings $a_1a_2a_3 \cdots a_n$ and $b_1b_2b_3 \cdots b_p$, determine the largest r such that $a_1a_2a_3 \cdots a_r$ is a substring of $b_1b_2b_3 \cdots b_p$ for $1 \leq r \leq p$. (Incidentally, an algorithm for this problem can be used to compute the *Failure-Link* function for substring recognition[Knut74]).
6. *Common Substrings Problem* : Given two strings A and B, determine the strings that are common substrings of A and B [Aho74].
7. *Common Subsequences Problem* : A *subsequence* is a string resulted by deleting zero or more symbols (not necessarily consecutive ones) from a string. E.g. *toy* is a subsequence of *topology*. The Longest Common Subsequence (LCS) problem is to find a longest subsequence (or to determine its length) from two (or more) strings [Chav72, Aho74b, Chav75, Hirs75,77].
8. *Common Supersequence Problem* : The Shortest Common Supersequence (SCS) is an inverse version of the LCS problem: given two strings, determine a common supersequence which is of

the smallest length [Maie78].

1.2. Previous Results

The known results for the string matching problems are scattered in the literature. The following ones are more relevant to our study here. Our criteria for selections are based on computational complexities.

Morris and Pratt [Morr70] proposed an elegant sequential algorithm for the *Substring Matching Problem* whose linear time bound was formally established in [Knut74]. The algorithm is optimal in the sequential sense. Karp, Miller, and Rosenberg [Karp72] proposed sequential solutions to the *Repeated Substring Problem*. Let M and N denote the lengths of the two strings under matching. Without loss of generality, we assume that $M < N$. Hirschberg [Hirs74] proposed an $O(M*N)$ time algorithm for the LCS problem, which uses only $O(\text{Max}(M,N))=O(N)$ space. In [Hirs77], the upper bound for the LCS problem was improved to $O(r * N)$ time/space, where r denotes the length of the computed LCS. Clearly $r \leq M$. Hunt and Szymanski [Hunt77] proposed another approach for the LCS problem which considers only matches between two strings. The algorithm has a time complexity of $O(P \log N)$, where P is the number of "matches" between the two strings. I.e., let S_i denote the i 'th symbol in string S , and A, B denote the two strings involved. P is the number of instances of all (i, j) such that $A_i = B_j$. Their algorithm is very efficient under the "sparsely-matched" condition, i. e. when $P \ll M*N$, which is fairly common in practice. However, since $P = O(M*N)$, the worst-case complexity is not improved.

In [Hsu84], we presented two improved versions for the above two algorithms separately while still preserving the desirable features of both algorithms. Both algorithms achieved the similar worst-case time bounds of $O(r*M*\text{Max}(1, \log(N/M)))$ and $O(r*M*\text{Max}(1, \log(N/r)))$ respectively. Note that, previously, the worst-case complexity grows with N (the length of the longer string) rather than M (length of the shorter string). The complexity of the second algorithm can also be expressed as $O(P)$, which improves the $O(P \log N)$ complexity for the sparse case. Up to present, the other better bound for the LCS problem is due to Masek and Paterson [Mase80]. They extended the approach of Arlazarov et. al. [Arla70] in multiplying Boolean matrices (the "Four-Russian Algorithm") to give an $O(N*M/\log N)$ time algo-

rithm for the LCS problem. However, because of the extensive preprocessing required, the constant factor involved in the complexity of the algorithm is much greater than those of the other algorithms. Therefore, unless the problem size is extremely large, the algorithms can not outperform the other algorithms.

While currently the solutions to the string processing problems are rather unrelated, Section II shows a framework that can be used to coherently design and analyze algorithms for the class of problems given in Section 1.2. In Section III, we show an efficient algorithm for solving the Maximal Common Substring problem. In Section IV, we show that the problems listed in Section 1.2 can be solved with the common framework and a similar procedure.

II. FORMULATION

2.1. Correlation Matrices and Lists

A string S is a sequence of symbols $S_1 S_2 S_3 \dots S_{|S|}$, where S_i denotes the i 'th symbol in S , and $|S|$ denotes the length of the string. The *Correlation* between two strings A and B is a binary relation $R_{A,B} = \{ (i, j) | A_i = B_j \}$. In other words, $R_{A,B}$ contains all the pairs of positions in A and B where two symbols match each other.

The *Correlation Matrix* associated with the strings A and B , denoted $M_{A,B}$, is defined as follows:

$$M_{A,B}(i, j) = 1 \quad \text{if } A_i = B_j; \\ = 0 \quad \text{otherwise.}$$

Example

Figure 1 shows the correlation matrix associated with $A = 'tcaggat'$ and $B = 'gatttatgcag'$. For convenience, we show only the "1" entries. Also, since A_i is associated with row i of the matrix (and B_j is associated with column j), we show the two strings along with the matrix.

	g	a	t	t	t	a	t	g	c	a	g	g
t			1	1	1		1					
c									1			
a		1				1				1		
g	1							1			1	1
g	1							1			1	1
a		1				1				1		
t			1	1	1		1					
t			1	1	1		1					

Figure 1. The Correlation Matrix for Strings A and B.

Fig. 1 also shows the row/column regularity typical of the correlation matrix. The following property enables us to represent the correlation between strings more efficiently.

Definition

$$Row(i) = \{ x \mid B_x = A_i \}$$

$$Col(j) = \{ x \mid B_x = B_j \}$$

The following property is obvious.

Property I

$$Row(i) = Row(j) \text{ if and only if } A_i = A_j$$

$$Col(i) = Col(j) \text{ if and only if } B_i = B_j$$

In the *correlation list* representation, a "1" entry at (i, j) of $M_{A,B}$ is represented by an entry j in a list corresponding to row i.

Example

Let String A='ggtaaatta', and B='aagggttc'. The following shows the list representation for the correlation between two strings.

CorrelationLists:

List('a'): Row(1)=Row(2)={ 4, 5, 6, 9 }.

List('c'): Row(7)=nil.

List('g'): Row(3)=Row(4)={ 1, 2 }.

List('t'): Row(5)=Row(6)={ 3, 7, 8 }.

Figure 2 Correlation Lists

Assume that the alphabet (i.e. the set of all permissible symbols) is known beforehand, it is relatively easy to construct the correlation lists in $O(M+N+s)$ time [Hsu84], by using the symbols as indices to an array, where s denotes the size of the alphabet, and M, N denote the lengths of the input strings.

The following property of the correlation matrix is convenient for identifying the common substrings.

Property II

A series of "1" entries on the correlation matrix $M_{A,B}$, (i_1, j_1) , (i_2, j_2) , (i_3, j_3) , correspond to a common substring of A and B, if and only if $i_{t+1} = i_t + 1$ and $j_{t+1} = j_t + 1$ for $t \geq 1$.

[proof]

(Obvious. See Fig. 3 for illustration.)

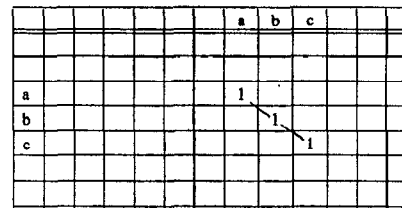


Figure 3 Common Substring and 1-Chain.

A series of "1" entries satisfying Property 1 will be called a *1-chain*, which may be identified with its *head* and *tail*, which are coordinates of the two end points on the matrix. Since we are interested in CS which are of maximal lengths, we will consider 1-chains which are of maximal lengths, i.e., the ones which are not contained in the others. (See Figure 4 for illustration.)

Let $CS(i)$ denote the CS represented by a 1-chain i . Let $I_{A,B}$ denote the set of such maximal 1-chains associated with strings A and B, and $CS(I_{A,B}) = \{ CS(i) \mid i \text{ in } I_{A,B} \}$. It is not difficult to see that the set of all maximal CSs are a subset of $CS(I_{A,B})$. (We will henceforth drop the subscript for convenience.)

Lemma A

MCS is a subset of $CS(I)$

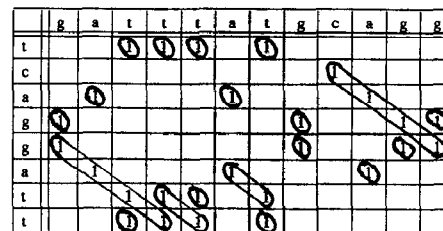


Figure 4 Maximal 1-Chains on Correlation Matrix

2.2. A Scheme for Computing MCS

Based on Lemma A, our general approach to determine the MCSs is as follows.

Step 1: Generate the set of all maximal 1-chains.

Step 2: Generate the set of common substrings which correspond to the 1-chains obtained in Step 1.

Step 3: Eliminate the CS which are non-maximal from the candidate set from Step 2.

For Step 2, we observe that two 1-chains may correspond to the same CS, so the equivalence classes have to be identified. See Fig. 4 for example. (The equivalence relation "==" can be defined as follows: Let i_1, i_2 be two 1-chains in I , $i_1 == i_2$ if and only if $CS(i_1) = CS(i_2)$.) For this purpose (also for determining the maximal CS from $CS(I)$), we define the following relation "<<" on I .

Definition

Let (h_j, t_j) and (h_i, t_i) respectively denote the head and tail of Chain j . Let i_1, i_2 denote two 1-chains in I . $i_1 << i_2$, if and only if one of the following conditions holds:

- i). $h_{21} \leq h_{11} \leq t_{11} \leq t_{21}$ ("horizontal dominance");
- ii). $h_{22} \leq h_{12} \leq t_{12} \leq t_{22}$ ("vertical dominance").

In either case, i_1 is said to be dominated by i_2 .

Lemma B

Two Chains i_1 and i_2 are equivalent if and only if there exists an i_3 such that $i_1 << i_3 << i_2$ and $i_2 << i_3 << i_1$, where i_3 may or may not be equal to either i_1 or i_2 .

Example

Figure 5 shows a graph representation of the "dominance" relation on a set of maximal 1-chains, where nodes correspond to the chains, and an edge from i_1 to i_2 represent that $i_1 << i_2$. Note that, in the graph, we intentionally preserve the locations of the chains on the correlation matrix to show that equivalent 1-chains dominate each other under the transitive closure of "<<". (In other words, nodes in a cycle of the dominance graph G constitute an equivalence class). We will determine the equivalence classes based on this special geometric property.

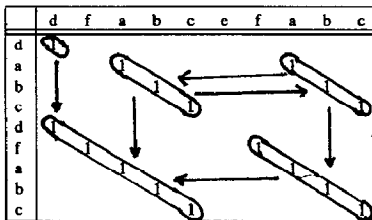


Figure 5. Dominance between Maximal 1-Chains

Let E denote the collection of all of the equivalence classes. For convenience, we write $CS(e) = C$, if e denote an equivalence class of 1-chains which all correspond to a common substring C .

Lemma C

$CS(e)$ is maximal if and only if, for each i in e , there is no $i' \notin e$ such that $i << i'$.

[proof]

The following two observations are essential: (c.f. Fig. 5)

- i). If $CS(e)$ is maximal and $i \in e$, then there is no $i' \in I$, $i' \notin e$, such that $i << i'$.
- ii). If $CS(e)$ is non-maximal, then there exists at least an $i' \notin e$, such that i' dominates one of the i in e . Q.E.D.

Alternatively, let E denote the set of equivalence classes of the nodes on G . We define a relation " $\leftarrow$ " on E as follows.

$e_1 \leftarrow e_2$, if and only if

- i). $e_1 = e_2$, and
- ii). there is an $i_1 \in e_1, i_2 \in e_2$, such that $i_1 << i_2$.

Clearly, by Lemma C, the maximal CS are those not dominated in " $\leftarrow$ ". The following is a simpler version of Lemma C.

Corollary

$CS(e)$ is maximal if and only if there is no e' in E such that $e' \leftarrow e$.

[Proof] (from our construction).

Example

A reduced graph G' for the dominance graph G given in the previous example is shown in Fig. 6. G' can be obtained by replacing nodes of an equivalence class with a single node, and by preserving the connections between different nodes in G . We note that G' is acyclic. In terms of graphs, the maximal ones correspond to the terminal nodes on the graph G' .

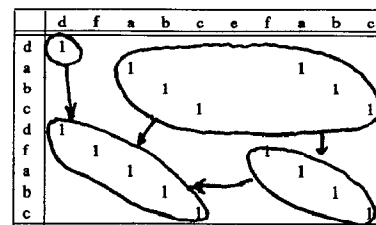


Figure 6. Reduced Dominance Graph (Derived from Fig. 5)

III. DATA STRUCTURES AND ALGORITHMS

The following process is presented for generating the maximal common substrings of two given strings:

Phase 1: Determine I, the set of maximal 1-chains for the given input strings.

Phase 2: Determine equivalent classes of I, based on Lemma B.

Phase 3: Determine CS which are maximal based on Lemma C.

3.1. Phase 1

We present two different approaches for identifying the 1-chains of maximal lengths. A 1-chain will be characterized by a triple, which consists of a unique "id", its "head" (a coordinate), and its "tail" (a coordinate). It gets its id when the 1-chain is first encountered in the algorithm. A variable "Count" will be used to keep track of the total number of 1-chains encountered so far, whose value is then assigned as an id to the new 1-chain.

i). Algorithm A

The first algorithm is based on a matrix representation of the id's. Let ID be an $M \times N$ array. ID[i,j] records the id of the 1-chain passing through the entry at (i,j). The following Pascal-like paragraph describes the simple algorithm which works row-by-row. The key point of this scheme is that a "1" entry on $M_{A,B}$ either extends an existing 1-chain from the previous row, or it initiates a new 1-chain.

(*Initialization*)

For Row:= 0 to M Do ID[Row, 0] := 0;

For Col:= 0 to N Do ID[0, Col] := 0;

Count :=0;

For Row := 1 to M Do

For Col := 1 to N Do

If A[Row] = B[Col]

Then (* we have a "1" entry here *)

If ID[Row-1, Col-1] <> 0

then (* extend the old chain *)

Begin

ID[Row, Col] := ID[Row-1, Col-1];

Tail(ID[Row, Col]) <== (Row, Col)

End

else (* this is a new 1-chain *)

Begin

Count := Count + 1;

ID[Row, Col] := Count;

Head(Count) <== (Row, Col);

Tail(Count) <== (Row, Col)

End

Since there are $M*N$ entries on the matrix, and for each entry, exactly one comparison is made, the algorithm takes $\Theta(M*N)$ execution time. The space requirement is also bounded by $\Theta(M*N)$. Since there could be $\Theta(M*N)$ "1" entries and as many 1-chains in the worst case, the above simple algorithm is already optimal in such cases.

However, usually the strings are sparsely matched [Aho74]. In other words, in most cases, there are only $O(M+N)$ "1" entries on the correlation matrix. So, as will be shown next, we can often do better than implied by the quadratic bound.

ii). Algorithm B

In algorithm A, the representation of the $M*N$ matrix is the bottleneck of the approach. Algorithm B will work by scanning the correlation list corresponding to each row. We keep an ordered list of the "tails" of the 1-chains ("active chains") which are to be extended at the row. Let the tails be given by the following sequence: $T=t_1 < t_2 < t_3 < \dots < t_l$. We also keep an ID field to identify the active 1-chains. Recall that a "1" entry either extends an existing chain or initiates a new chain. Let X denote the correlation list at the row being considered: $X=x_1 < x_2 < x_3 < \dots < x_r$. Then, to determine whether an x_k in X can extend an existing chain (represented by the tail), it suffices to search in the list T. To avoid inefficient searches, the following observations are crucial:

1). Elements in X and T form an ordered list,

2). The search is required for *every* element of X.

Thus, if we combine the individual searches, we are essentially *merging* the two lists X and T.

The following pseudo-code is an algorithmic description of the above scheme.

Count :=0;

Next-T <== nil;

```

For Row := 1 to M Do
  Begin
    X := the correlation list for the current row
    T := Next-T;
    (* T denotes the list of tails of the active chains*)
    Z <== Merge(T, X);
    (*Let Z = z1, z2, ..., zw *)
    For each element zi in Z Do
      Case zi is from X :
        If (zi - zi-1) = 1 and zi-1 is from T
        then (* extend the existing chain*)
          Begin
            (*Let ID denote the id of the chain i*)
            Tail(ID) <== (Row, zi);
            Next-T <== Append ( zi , ID)
          End
        else (* initiate a new chain *)
          Begin
            Count := Count + 1;
            ID := Count;
            Head(ID) := (Row, zi);
            Tail(ID) := (Row, zi);
            Next-T <== Append (zi, ID)
          End
      Case zi is from T: (*do nothing*)

    End (*process at the row*)

```

Lemma D

Assuming known alphabet, the common substrings of two strings of length M and N can be determined in $O(P+M+N+t)$ steps, which includes the preprocessing time to set up the correlation lists. Where M, N denote the lengths of the strings involved, P denotes the occurrences of matches between the two strings, and t denotes the size of the alphabet.

[proof]

By using the Lincar (tape) Merge algorithm, which can merge two ordered lists in linear time, we may perform the chaining efficiently. Note that the "tails" of the active 1-chains at a row are exactly the "1" entries of the previous row. Thus the "1" entries on each row participates no more than two times in the merging process. (Row i-1 "merges" with row i at step i; Row i again "merges" with row i+1 at step i+1). Therefore, the total time can be bounded by $O(n_1+n_2+n_3+\dots+n_m)=O(P)$, where n_i denotes the number of the "1" entries in row i, and P denotes the total number of them. Q.E.D.

3.2. Phase II

By Lemma B, we can determine whether two chains i_1, i_2 are equivalent by testing the (horizontal and vertical) dominance conditions. In our head-tail representation of the 1-chains, we can perform this rather efficiently, without doing symbol-by-symbol comparisons on $CS(i_1)$ and $CS(i_2)$.

Lemma E

Given I, the set of equivalence classes can be determined in $O(III)$ time.

[proof]

Briefly, the following steps suffice,

- 1). Repeat Steps 2 and 3 for two coordinate axes.
- 2). Project the 1-chains onto the selected axis, which become a collection of line segments. Sort the end points of the segments.
- 3). Report the ones having the same beginning and ending points. (They are equivalent, by Lemma B).
- 4). Determine the equivalence classes (by using the well-known set union algorithm in [Tarj75].)

For notational convenience, let $X := III$, the number of maximal 1-chains considered. Step 2 takes at most $O(X \log X)$ time. Step 3 takes at most $O(X)$ time. Step 4 takes $O(XG(X))$ time, where $G(X)$ is an extremely slow increasing function of X . The total amount of time is therefore bounded by $O(X \log X)$. By simple summing, the space requirement can be bounded by $O(III)$. Q.E.D.

3.3. Phase III

Let L denote the sorted list of the segments obtained in a projection (c.f. Phase II). Based on Lemma C, the following scheme is presented for determining the the maximal CS.

Maintain a list S while scanning L. S is to keep track of the segments being considered. We include a segment when its first end point is encountered, and delete it when its second end point is encountered. Now, if a sement s_1 is contained in another segment s_2 , s_1 will be appended to S later than s_2 , while s_1 will be deleted earlier than s_2 . Thus it is a simple matter to detect dominance.

Lemma F

Given the 1-chains, the set of maximal common substrings, MCS(I), can be generated in $O(|I| \log |I|)$ time, using at most $O(|I|)$ space.

Theorem 1

Given two strings of length M and N respectively, the common substrings can be computed in $O(M+N+t \cdot P \log P)$ time, where t denotes the size of the alphabet and P denotes the number of matches between the two strings.

[proof]

Based on Lemmas A-F, and that $|I| \leq P$. Q.E.D.

IV. DISCUSSION

In Phase II of our algorithm, we have suggested the use of the linear merge algorithms. For theoretical interests, we may well take advantage of any good merging algorithm. For example, the Hwang-Lin Merge has an $O(M \cdot \max(1, \log N/M))$ asymptotic bound (assume that $N > M > 0$), which is desirable when the two lists under merging have disparate lengths. On the other hand, it has been shown that the linear merge algorithm is already optimal for two sequences with similar lengths.

It is interesting to note that we may also use the correlation matrix and correlation lists for solving the subsequence problems. The idea is based on the following property.

Property III

A series of "1" entries on the correlation matrix, $(i_1, j_1), (i_2, j_2), (i_3, j_3), \dots$ corresponds to a common subsequence of A and B , if and only if $i_{t+1} > i_t$ and $j_{t+1} > j_t$ for $t \geq 1$.

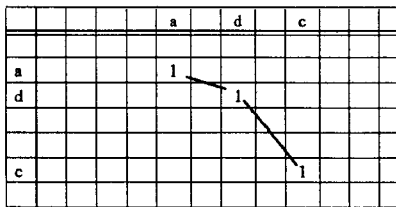


Figure 7. Identifying Common Subsequence on Matrix

In [Hsu84], given the correlation list representation, the concept of merging was also applied to implement a row to row operation for solving the LCS problem. Thus, the substring problems and the subsequence problems, which are traditionally treated as very different problems, can now be solved with a common framework. The problems can also be solved using similar procedures.

Furthermore, since merging algorithms have been well known and have been optimized for daily uses, our reduction of the string problems into merging have certain advantages:

- 1). The optimized code need only be modified slightly to implement our algorithms.
- 2). Any new improvement(e.g. parallel algorithms) for the general merging problem automatically translates to improvement for the string processing.

Very few results have been obtained for the generalization of the two-string problems. With few exceptions, all of the existing approaches are for the two-string case, which turn out to be inefficient when extended to the multi-string case. In [Itog82], the LCS problem and the SCS problem are extended to the multi-string cases. An algorithm is also provided which takes $O(k \cdot |S_1| \cdot |S_2| \cdot |S_3| \cdot \dots \cdot |S_k|)$, where $|S_i|$ denotes the length of the i 'th string. In [Hsu84b], we presented a new general approach for computing an LCS for a set of k strings, which is later shown to be adaptable to solve the other problems [Hsu87]. Assuming a finite symbol set, it is shown that the presented method requires a preprocessing time $O(|S_1| + |S_2| + \dots + |S_k|)$, and a processing time $O(P)$ where P denotes the number of matched positions among these strings. Since P is typically a very small fraction of the $|S_1| \cdot |S_2| \cdot |S_3| \cdot \dots \cdot |S_k|$, the proposed method is a significant improvement.

V. CONCLUSION

By use of the correlation matrix and the correlation lists, we have shown that the common substring problems can be solved with a common framework. We have also presented algorithms for solving the MCS problem in $O(\text{Max}(M, N, t, P \log P))$ time, using $O(\text{Max}(M, N, t, P))$ space. Since P is often a linear function of M and N , the proposed algorithm may be considered as very efficient.

VI. REFERENCES AND BIBLIOGRAPHY

- [Aho76] A. V. Aho, D. S. Hirschberg, and J. D. Ullman, Bounds on the Complexity of the Maximal Common Subsequence Problem, *J. ACM*, 23(1976), pp. 1-12.
- [Aho74] A. V. Aho, J. E. Hopcroft, and J. D. Ullman, "The Design and Analysis of Computer Algorithms," Addison-Wesley, Reading, MA, 1974.
- [Aho74b] A. V. Aho, D. S. Hirschberg, and J. D. Ullman, Bounds on the Complexity of the Maximal Common Subsequence Problem, in *Proceedings, 15th IEEE Symposium on Switching and Automata Theory*, 1974, pp. 104-109.
- [Arla70] V. L. Arlazarov, E. A. Dinic, M. A. Kronrod, and I. A. Faradzev, On Economic Construction of the Transitive Closure of a Directed Graph, *Dokl. Akad. Nauk SSSR* 194(1970), pp. 487-488 [Russian]; *Soviet Math. Dokl.* 11(1970), 1209-1210.
- [Chan86] S. K. Chang and E. Jungert, A Spatial Knowledge Structure for Image Information Systems Using Symbolic Projections, *Fall Joint Comput. Confer.*, Dallas, TX, 1986, pp. 79-86.
- [Chan86b] S. K. Chang et. al., Iconic Indexing by 2D Strings, *Proc. of IEEE Workshop on Visual Languages*, Dallas, TX, 1986.
- [Chav72] V. Chavatal, D. A. Klarnet, and D. E. Knuth, Selected Combinatorial Research Problems, *TR STAN-CS-72-292*, P.26, Stanford Univ., 1972.
- [Chav75] V. Chavatal and D. Sankoff, Longest Common Sequences of Two Random Sequences, *TR STAN-CS-75-477*, Stanford Univ., 1975.
- [Dayh65] M. O. Dayhoff, Computer Aids to Protein Sequence Determination, *J. Theoret. Biol.* 8(1965), pp. 97-112.
- [Dayh69] M. O. Dayhoff, Computer Analysis of Protein Evolution, *Sci. Amer.* 221(1969), 86-95.
- [Dewa82] R. B. K. Dewar, S. M. Merritt, and M. Sharir, "Some Modified Algorithms for Dijkstra's Longest Upsequence Problem," *Acta Informatica*, 18 (1982), pp. 1-15.
- [Fred75] M. L. Fredman, On Computing Length of the Longest Increasing Subsequences, *Discrete Math.* 11(1975), pp. 29-36.
- [Free61] H. Freeman, On the Encoding of Arbitrary Geometric Configurations, *IEEE Tran. Comput.*, vol. EC-10 (1961), pp. 260-268.
- [Fu73] K. S. Fu and B. K. Bhargava, Tree Systems for Syntactic Pattern recognition, *IEEE Tran. Comput.*, 20 (1973), pp. 1087-1099.
- [Fu74] K. S. Fu, *Syntactic Methods in Pattern Recognition*, Academic Press, NY, 1974.
- [Fu77] K. S. Fu, *Syntactic Pattern Recognition, Applications*, Springer-Verlag, NY, 1977.
- [Fu81] K. S. Fu, Recent Progress in Syntactic Pattern Recognition, in *Progress in Pattern Recognition*, L. N. Kanal and A. Rosenfeld (editors), North-Holland Publishing Co., 1981.
- [Gala80] J. Gallant, D. Maier, and J. A. Storer, On Finding Minimal Length Superstrings, *J. Comput. Sys. Sci.* 20 (1980), pp. 50-58.
- [Hirs75] D. S. Hirschberg, A Linear Space Algorithm for Computing Maximal Common Subsequences, *J. ACM*. 24 (1975), pp. 341-343.
- [Hirs75b] D. S. Hirschberg, The Longest Common Subsequence Problem, Ph.D. Thesis, Princeton Univ., N. J., 1975.
- [Hirs77] D. S. Hirschberg, Algorithms for the Longest Common Subsequence Problem, *J. ACM* 24 (1977), pp. 664-675.
- [Hsu84] W. J. Hsu and M. W. Du, New Algorithms for the LCS Problem, *J. Comput. Sys. Sci.*, 29 (1984), pp. 133-152.
- [Hsu84b] W. J. Hsu and M. W. Du, Computing a Longest Common Subsequence for a Set of Strings, *BIT*, 24 (1984), pp. 45-59.
- [Hsu87] W. J. Hsu, Algorithms for a Cluster of String Matching Problems, (in preparation)
- [Hsu87b] W. J. Hsu, Symbol Matching on a Mesh-Connected Computer, (in preparation)
- [Hunt75] J. W. Hunt and M. D. McIlroy, An Algorithm for Differential File Comparison, *Comput. Sci. TR* 41, 1975.
- [Hunt77] J. W. Hunt and T. G. Szymanski, A Fast Algorithm for Computing Longest Common Subsequences, *Comm. ACM*, 1 (1977), pp. 350-353.
- [Itog81] S. T. Itoga, The String Merging Problem, *BIT* 21 (1981), pp. 20-30.
- [Knut74] D. E. Knuth, J. H. Morris, and V. R. Pratt, Fast Pattern Matching Algorithms, *STAN-CS-74-440*, Stanford Univ., Stanford, 1974.
- [Lowr75] R. Lowrance and R. A. Wagner, An Extension of the String to String Correction Problem, *J. ACM*, 22 (1975), pp. 177-183.
- [Lu78] S. Y. Lu and K. S. Fu, A Sentence-to-Sentence Clustering Procedure for Pattern Analysis, *IEEE Tran. System Man Cybernet.*, SMC-8 (1978), pp.381-389.
- [Maie78] D. Maier, The Complexity of Some Problems on Subsequences and Supersequences, *J. ACM*, 25 (1978), 322-336.
- [Mase80] W. J. Masek and M. S. Patterson, A Fast Algorithm for Computing String Edit Distances, *J. Comput. Sys. Sci.*, 20 (1980), pp. 18-31.
- [Mode84] J. L. Modelevsky, Computer Applications in Applied Genetic Engineering, *Advances in Applied Microbiology*, vol. 30, 1984, pp.169-195.
- [Morg70] H. L. Morgan, Spelling Correction in Systems Programs, *Comm. ACM*, 13 (1970), pp.90-94.
- [Morr70] J. H. Morris and V. R. Pratt, A Linear Pattern Matching Algorithm, *TR-40*, Comput. Cent., U. of California, Berkeley, CA, 1970.
- [Mukh80] A. Mukhopadhyay, A Fast Algorithm for the Longest-Common-Subsequence Problem, *Inf. Sci.* 20 (1980), pp. 69-82.

[Need70] S. B. Needleman and C. S. Wunsch, A General Method Applicable to the Search for Similarity in the Amino Acid Sequence of Two proteins, *J. Molecular Biol.* 77 (1970), pp. 443-453.

[Pavl82] T. Pavlidis, Algorithms for Graphics and Image Processing, Computer Science Press, Inc., Rockville, MD, 1982.

[Robe85] Y. Robert and M. Tchuente, "A Systolic Array for the Longest Common Subsequence Problem," *Info. Proc. Lett.*, 21 (1985), pp. 191-198.

[Sanc85] R. Sanchez-Pescador, et. al., Nucleotide Sequence and Expression of an AIDS-Associated Retrovirus(ARV-2), *Science*, vol. 227 (1985), pp. 484-492.

[Sank72] D. Sankoff, Matching Sequences Under Deletion Insertion Constraints, *Proc. Nat. Acad. Sci. U.S.A.* 69 (1972), pp. 4-6.

[Sank73] D. Sankoff and R. J. Cedergren, A Test for Nucleotide sequence Homology, *J. Molecular Biol.*, 77 (1973), pp. 159-164.

[Selk73] S. M. Selkow, The Tree-to-Tree Editing Problem, *Inform. Process. Lett.* 6 (1973), pp. 184-186.

[Sell74] P. H. Sellers, An Algorithm for the Distance between Two Finite Sequences, *J. Combin. Theory Ser. A* 16 (1974), pp. 253-258.

[Tarj75] R. E. Tarjan, Efficiency of a Good but not Linear Set Union Algorithm, *J. ACM* 22 (1975), pp. 215-225.

[Wagn74] R. A. Wagner, Common Phrases and Minimum-Space Text Storage, *Comm. ACM*, 16 (1974), pp. 148-152.

[Wagn75] R. A. Wagner and M. J. Fischer, The String-to-String Correction Problem, in *Proceedings, 7th Ann. ACM. on Theory of Comput.* 1975, pp. 218-223.

[Wagn75b] R. A. Wagner, On the Complexity of the Extended String-to-String Correction Problem, *Proc. Seventh Ann. ACM Symp. on Theory of Comput.*, (1975), pp. 218-223.

[Wong76] C. K. Wong and A. K. Chandra, Bounds for the String Editing problem, *J. ACM*, 28 (1976), 13-18.